

Sales Prediction Management System

Machine Learning Study Guide

Linear Regression · LSTM · Train vs Predict · Metrics · Your Project

A plain-English learning note for the thesis project.

Read slowly. Practice the quizzes. Connect each idea to the code files listed.

How to use this PDF

1) Read sections 1–3 for the big picture. 2) Study Linear Regression, then LSTM. 3) Learn MAE/RMSE with the shop example. 4) Map ideas to your project files. 5) Try the self-check questions without looking at answers first.

1. What is prediction?

Prediction (also called **forecasting** when talking about the future) means: looking at **what already happened** and estimating **what might happen next**.

In this project, the question is: *For each product, about how many units will we sell in the next few days?*

It is **not magic** and **not a guarantee**. It is a smart estimate the computer learns from patterns in historical sales.

Everyday shop example

- Randomie often sells more on weekends.
- When there is a promo, sales jump.
- The model learns: weekend + promo → higher sales.
- Then it says: next 7 days, expect about X cartons per day.

Key words

History = past sales in MySQL. **Pattern** = weekend/month/promo/trend. **Forecast** = estimated future quantities. **Model** = the learned rule/brain that maps patterns → forecast.

2. The prediction pipeline (factory line)

Your system does not jump straight to AI. It follows a clear pipeline:

MySQL sales → clean daily table → add features → train models → save files
→ API predicts

Step	What happens	Why it matters
1. Collect	Read sales from the database	No data = nothing to learn
2. Clean	One quantity per product per day; missing days = 0	Models need a regular timeline
3. Features	Add day-of-week, month, weekend, promo flags	Gives the model useful clues
4. Train	Learn Linear Regression + LSTM	Creates the 'brain' files
5. Evaluate	Compute MAE and RMSE on held-out days	Shows how wrong the guesses are
6. Predict	Forecast next N days via Flask API	Useful output for managers

3. Features (the clues)

A model does not only see 'quantity sold'. We also give it **features** — extra clues that help explain why sales go up or down.

Feature	Meaning	Example
qty (past)	Recent sales amounts	Last 30 days of units sold
day_of_week	Mon=0 ... Sun=6	Weekends often busier
month	1–12	Some months are peak season
is_weekend	1 if Sat/Sun else 0	Simple weekend flag
is_promo	1 if promotion active	From Promotions table
day_of_year	1–365/366	Fine-grained calendar position

In code: `ml_service/preprocess.py` builds the daily series and these features. This also supports your thesis objective about external factors (promo + seasonality).

4. Linear Regression — the simple student

4.1 What it is

Linear Regression is a classical statistics / machine-learning method. It finds a relationship that looks like a weighted sum of clues:

$$\text{predicted_sales} \approx a + b_1 \cdot x_1 + b_2 \cdot x_2 + b_3 \cdot x_3 + \dots$$

- **y** (predicted_sales) = the answer we want
- **x1, x2, ...** = features/clues (past sales window, weekend, promo...)
- **a** = intercept (starting baseline)
- **b1, b2, ...** = weights the model learns (importance of each clue)

4.2 Everyday picture

Imagine plotting points for 'was it weekend?' vs 'sales', or 'promo?' vs 'sales', and drawing the **best straight-line style rule** through the cloud of points. That rule is the model.

Example intuition (not exact numbers):

- Base daily sales $\approx$ 4 units
- If weekend, add about +2
- If promo, add about +3
- If recent days were high, add a bit more

4.3 Why we use it in SPMS

- Easy to explain in a thesis defense
- Trains very fast
- Good **baseline** (simple method first, then compare with LSTM)
- Matches your write-up mention of regression models

4.4 Strengths and weaknesses

Strengths	Weaknesses
Simple, transparent, fast	Struggles with complex wave-like patterns over long time
Great teaching / comparison baseline	Assumes roughly linear combinations of features
Works with fewer data points than deep nets	May underfit rich seasonal dynamics

4.5 How YOUR project uses Linear Regression

- File: **ml_service/models_lr.py**
- Looks at a flattened window of the last ~30 days of features
- Trains with scikit-learn **LinearRegression**
- Saves model to **models/lr_product_X.joblib**
- Predict rolls one day at a time for the next 7 days (horizon)

Train vs Predict (Linear Regression)

Train: show many past windows + actual next-day sales; learn weights a, b . **Predict:** take the newest window and apply the learned weights to estimate future days.

4.6 Mini math you should know

During training, the model chooses weights so that the average error between predicted y and true y becomes small. A common training idea is minimizing squared error:

$$\text{error} = (\text{true_sales} - \text{predicted_sales})^2 \rightarrow \text{make this small on average}$$

You do not need to derive proofs for the project, but you should be able to say: Linear Regression fits a weighted sum of features to minimize prediction error.

5. LSTM — the memory student

5.1 What the name means

LSTM = Long Short-Term Memory. It is a type of **neural network** (deep learning model) designed for sequences — data that comes in order over time.

Sales is a sequence: Day1 → Day2 → Day3 → ... Shuffling those days would destroy the story. LSTM reads the story in order.

5.2 Neural network in one breath

A neural network is many small math units (neurons) arranged in layers. Each connection has a weight. Training adjusts weights so outputs get closer to truth. Deep learning just means networks with enough structure to learn complex patterns.

5.3 Why 'memory'?

Older simple networks forget long context quickly. LSTM cells use internal gates (forget / input / output style mechanisms) so the model can:

- Keep useful recent information (short-term)
- Also carry important longer patterns (long-term-ish memory)
- Ignore noise that is not helpful

You do not need to implement gates by hand for the thesis oral. You should say: LSTM is a sequence model with memory, good for time-series sales.

5.4 Everyday picture

Linear Regression is like a checklist of rules. LSTM is more like a staff who watched the counter for weeks and says:

“Sales climbed for 10 days, last weekend was high, promo just ended, so the next few days may stay medium-high then cool down.”

5.5 Sliding window (very important)

In your project, lookback is about **30 days** and horizon about **7 days**:

- Input: features for the last 30 days
 - Output: estimate for the next day (then roll forward for 7 days)
 - This is called a sliding / rolling window approach
- Example: use days 1–30 to guess day 31; then use days 2–31 to guess day 32; and so on.

5.6 Strengths and weaknesses

Strengths	Weaknesses
Good at time patterns and sequences	Needs more history than simple models
Can capture non-linear ups and downs	Harder to explain than a straight formula
Matches modern ML literature / your write-up	Can overfit or fail if data quality is poor

5.7 How YOUR project uses LSTM

- File: **ml_service/models_lstm.py**
- Uses **PyTorch** LSTM layers when available
- Fallback NumPy LSTM exists if Torch is missing
- Saves to **models/lstm_product_X.joblib**
- API default model is often LSTM, with linear as baseline comparison

Defense sentence you can memorize

"We use Linear Regression as a transparent baseline and LSTM as a sequence-aware deep model. Both are trained on chronological sales with calendar and promotion features, and compared using MAE and RMSE."

6. Linear Regression vs LSTM (side by side)

Point	Linear Regression	LSTM
Type	Classical ML / statistics	Deep learning neural net
Thinks in	Weighted sum of clues	Sequence + memory over days
Speed	Very fast	Slower to train
Explainability	High	Medium
Best for	Baseline, simple trends	Wavy multi-day patterns
Your file	models_lr.py	models_lstm.py
Your role	Comparison baseline	Main advanced model

7. Train vs Predict (both models)

7.1 Train = learning time

- Show the model many past examples
- Model adjusts internal numbers to reduce error
- We hold out the latest part of the timeline as a test set (no shuffling)
- Command: **python train.py**
- Writes metrics into **model_runs** table and saves .joblib files

7.2 Predict = exam time

- Ask a new question: product_id + next 7 days
- Load the saved model file
- Return forecast dates + quantities as JSON
- Endpoint: **POST /predict** on Flask (port 5000)

Never train and test on randomly shuffled time-series rows. Time has direction: train on earlier days, test on later days.

8. MAE and RMSE (how we score models)

After predicting, we compare guesses to real sales on the test days.

Metric	Plain meaning	Why useful
MAE	Mean Absolute Error = average of true – predicted	Easy: 'wrong by about N units on average'
RMSE	Root Mean Square Error = square errors, average, square-root	Punishes large mistakes more than MAE

Tiny numerical example

True sales for 3 days: 10, 8, 12. Predictions: 9, 10, 11.

- Absolute errors: $|10-9|=1$, $|8-10|=2$, $|12-11|=1$

- $MAE = (1+2+1)/3 = 1.33$ units

- Squared errors: 1, 4, 1 → mean 2 → $RMSE = \sqrt{2} \approx 1.41$

Lower MAE/RMSE is better. If LSTM MAE is 1.9 and Linear MAE is 2.3 on the same product, LSTM was closer on average for that test period.

9. How this maps to YOUR codebase

Idea	Where in the project
Shop UI (sell, stock, reports)	web/ modules (Phase 1 PHP)
Prediction Engine	ml_service/ Flask app (Phase 2)
Cleaning + features	ml_service/preprocess.py
Linear Regression	ml_service/models_lr.py
LSTM	ml_service/models_lstm.py
Training CLI	ml_service/train.py
API /health /predict /retrain	ml_service/app.py
Long demo history	ml_service/generate_seed_sales.py
Saved brains	ml_service/models/*.joblib
Stored forecasts / metrics	MySQL tables forecasts, model_runs

Remember the split: **PHP = shop for humans**, **Python = brain for forecasts**, **MySQL = shared notebook**.

10. Extra concepts worth learning

10.1 Time series

A time series is data recorded over time at regular intervals (daily sales). Special rule: respect order; do not randomly shuffle before train/test split.

10.2 Overfitting vs underfitting

- **Underfitting**: model too simple; misses real patterns (sometimes linear on complex waves)
- **Overfitting**: model memorizes training noise; looks great on train, weak on new days
- Holding out a chronological test set helps detect this

10.3 Baseline model

A baseline is a simple method you compare against. If a complex model cannot beat the baseline, the complexity may not be worth it. That is why Linear Regression stays in the project even when LSTM exists.

10.4 Horizon and lookback

- **Lookback**: how many past days the model reads (e.g. 30)
- **Horizon**: how many future days you ask for (e.g. 7)
- Longer horizon is usually harder and less accurate

10.5 External factors

Sales are not only 'yesterday's qty'. Promotions, seasonality, and weekends are external-ish signals. Your system includes calendar features and an is_promo flag from the promotions table — this supports Objective 2 in the write-up without needing live internet APIs.

10.6 Honest limits (say this in defense)

- Forecasts depend on history quality and length
- Sudden shocks (pandemic, strike) may not be predictable from old data
- Future promos help only if entered into the system
- A forecast supports decisions; it does not replace manager judgment

11. Glossary (quick revise before exams)

Term	Simple meaning
Feature	A clue/input the model uses
Label / target	The true answer during training (actual sales)
Model	Learned rule or network that maps features → prediction
Train	Learn from historical examples
Predict / infer	Use a trained model on new inputs
Baseline	Simple comparison model
LSTM	Sequence neural network with memory
Linear Regression	Weighted-sum formula model
MAE / RMSE	Error scores; lower is better
Chronological split	Train on earlier time, test on later time
API	Service endpoint other programs call (Flask)
JSON	Data format the API returns

12. Self-check quiz (try before looking at answers)

Questions

1. In one sentence, what does sales prediction mean in this project?
2. Why do we keep Linear Regression if we already have LSTM?
3. What is a feature? Name three features used in SPMS.
4. Why must we not shuffle time-series rows before train/test split?
5. What does MAE tell a manager in plain English?
6. What is lookback vs horizon?
7. Which project file cleans sales into a daily series?
8. Which side is the 'shop' and which side is the 'brain'?
9. Give one strength and one weakness of LSTM.
10. Why can forecasts still be wrong even with a good model?

Short answers

1. *Estimating future product sales quantities from historical patterns and related clues.*
2. *It is a simple, explainable baseline for comparison and thesis transparency.*
3. *A clue/input. Examples: past qty, day_of_week, is_promo (also month, weekend).*
4. *Time has order; random shuffle leaks future information into training.*
5. *On average, predictions are off by about MAE units.*
6. *Lookback = past days read; horizon = future days requested.*
7. *ml_service/preprocess.py*
8. *Shop = PHP web app; brain = Python Flask ml_service.*
9. *Strength: sequence memory / complex patterns. Weakness: needs more data; harder to explain.*
10. *Markets change; data can be incomplete; unexpected events are not in history.*

13. Practice plan (learn on your own)

- Day 1: Re-read sections 1–4. Explain Linear Regression to a friend in 60 seconds.
- Day 2: Re-read section 5. Draw a 30-day window → next-day arrow on paper.
- Day 3: Compute a tiny MAE by hand with 4 made-up days.
- Day 4: Open preprocess.py and models_lr.py; match comments to this PDF.
- Day 5: Open models_lstm.py and app.py; list the API endpoints from memory.
- Day 6: Run /health and one /predict (if services are up); interpret the JSON fields.
- Day 7: Write a half-page 'Methodology' paragraph using the defense sentence in §5.7.

Commands cheat-sheet

Generate history: `python generate_seed_sales.py --clear --days 300`

Train: `python train.py`

Start API: `python app.py`

Health: `http://127.0.0.1:5000/health`

Shop UI: `http://localhost/spms/`

End of study guide. Revisit before implementation demos and oral defense. Phase 3 will connect these forecasts to the manager dashboard charts.